



程序设计原理

字符变量与逻辑运算

任课教师：喻 梅
于瑞国
王建荣
李雪威
赵满坤



Data Type

Static vs Dynamic Data Types

How types are bound and checked strongly shapes what operations are valid and how fast programs run.

Aspect	C++ Static	Python Dynamic
Type Binding	Decided at compile time ; variables have fixed types.	Decided at runtime ; variables can reference any type.
Type Checking	Compile-time errors for invalid operations; earlier feedback.	Runtime errors if an operation is incompatible; later feedback.
Conversion	Controlled <i>implicit/explicit</i> casts; predictable semantics.	Frequent automatic coercion; convenient but may hide mistakes.
Performance	Optimized by the compiler; tight memory layout and speed.	Interpreter/VM overhead; flexible but generally slower.
Typical Pitfalls	Overly strict conversions; verbose type annotations.	Type surprises at runtime; latent bugs in untested paths.

- In static typing, the compiler knows the **shape of data** and the **valid operations** ahead of time.
- In dynamic typing, flexibility is traded for potential **runtime surprises**. 2



Data Type

A data type defines both **how data is stored** and **which operations** are valid on it

- Different data types support **different sets of operations**.
- Some data types can be **converted** to others, while some cannot.
- These choices affect **valid expressions** and their **semantics**.
- Today we focus on a special type `char` which behaves like both a **number** and a **symbol**.

“The `char` type is one of the most interesting types in C++ because it connects the numeric world with the textual world.”



ASCII Character Set, cont.

ASCII Character Set is a subset of the Unicode from \u0000 to \u007f

ASCII Quick Reference (A–Z, a–z, 0–9)

TABLE B.2 ASCII Character Set

	<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>
0	nul	soh	stx	etx
1	dle	dcl	dc2	dc3
2	sp	!	“	#
3	0	1	2	3
4	@	A	B	C
5	P	Q	R	S
6	‘	a	b	c
7	p	q	r	s

Group	Character	Decimal	Hexadecimal
A–Z	A	65	0x41
	M	77	0x4D
	Z	90	0x5A
a–z	a	97	0x61
	m	109	0x6D
	z	122	0x7A
0–9	0	48	0x30
	5	53	0x35
	9	57	0x39

<i>B</i>	<i>C</i>	<i>D</i>	<i>E</i>	<i>F</i>
vt	ff	cr	so	si
esc	fs	gs	rs	us
+	,	-	.	/
;	<	=	>	?
K	L	M	N	O
[	\	]	^	_
k	l	m	n	o
{		}	~	del

Note: 'A'–'Z' = 65–90, 'a'–'z' = 97–122, '0'–'9' = 48–57.

Difference between uppercase and lowercase letters: 'a' - 'A' = 32.



Character Data Type

- A char occupies **1 byte** at the underlying level and is an **integer type**.
- It has a **one-to-one correspondence** with **ASCII codes**.

So it can participate in **arithmetic** and **relational operations** such as **addition, subtraction, and comparison**.

```
char ch = 'A';  
  
cout << ch << " " << int(ch) << endl;  
  
ch++;  
  
cout << ch << " " << int(ch) << endl;
```

Note: 'A'-'Z' = 65-90, 'a'-'z' = 97-122, '0'-'9' = 48-57.

Difference between uppercase and lowercase letters: '**a**' - '**A**' = **32**.



Casting between char and Numeric Types

```
int i = 'a'; // Same as int i = (int)'a';  
char c = 97; // Same as char c = (char)97;
```

```
char c = 0xFF41;  
cout<<c<<endl;
```

```
char c = 65.25;  
cout<<c<<endl;
```

```
int i = 'A';  
cout<<i;
```



Escape Sequences for Special Characters

Character	Escape Sequence	Name	ASCII Code
\b		Backspace	8
\t		Tab	9
\n		Linefeed	10
\f		Formfeed	12
\r		Carriage Return	13
\\		Backslash	92
\'		Single Quote	39
\"		Double Quote	34

```
cout<<"He said \"programming is fun!\"<<endl;  
cout<<"\\t is a tab character"<<endl;  
cout << "A\tB\tC\n1\t2\t3\n";
```



Relation Operators

Operator	Name	Example	Result
<	less than	<code>1 < 2</code>	true
<=	less than or equal to	<code>1 <= 2</code>	true
>	greater than	<code>1 > 2</code>	false
>=	greater than or equal to	<code>1 >= 2</code>	false
==	equal to	<code>1 == 2</code>	false
!=	not equal to	<code>1 != 2</code>	true



The bool Data Type

Boolean value : *true* or *false*

A variable that holds Boolean value is known as a *Boolean variable*. The `bool` data type is used to declare Boolean variable.

For example:

```
bool lightOn = true;
```

Internally , C++ use 1 to represent true and 0 for false.

```
int a = 5, b = 3;  
bool b_test=5;  
  
cout << (a > b) << endl;  
cout << (a == b) << endl;  
cout << b_test << endl;
```



Logical Operators

<i>Operator</i>	<i>Name</i>	
!	not	logical negation
&&	and	logical conjunction
	or	logical disjunction



Logical Operators

p	!p
---	----

true	false
------	-------

false	true
-------	------

Example

!(1 > 2) is true, because (1 > 2) is false.

!(1 > 0) is false, because (1 > 0) is true.

p1	p2	p1 && p2
----	----	----------

false	false	false
-------	-------	-------

false	true	false
-------	------	-------

true	false	false
------	-------	-------

true	true	true
------	------	------

Example

(3 > 2) && (5 >= 5) is true, because (3 > 2) and (5 >= 5) are both true.

(3 > 2) && (5 > 5) is false, because (5 > 5) is false.



Logical Operators

p1	p2	p1 p2
false	false	false
false	true	true
true	false	true
true	true	true

Example

$(2 > 3) \parallel (5 > 5)$ is false, because $(2 > 3)$ and $(5 > 5)$ are both false.

$(3 > 2) \parallel (5 > 5)$ is true, because $(3 > 2)$ is true.



Operator Precedence

类别	运算符	结合性
后缀	() [] -> . ++ --	从左到右
一元	+ - ! ~ ++ -- (type)* & sizeof	从右到左
乘除	* / %	从左到右
加减	+ -	从左到右
移位	<< >>	从左到右
关系	< <= > >=	从左到右
相等	== !=	从左到右
位与 AND	&	从左到右
位异或 XOR	^	从左到右
位或 OR		从左到右
逻辑与 AND	&&	从左到右
逻辑或 OR		从左到右
条件	?:	从右到左
赋值	= += -= *= /= %= >>= <<= &= ^= =	从右到左
逗号	,	从左到右



Operator Precedence

```
#include <iostream>
```

```
int main() {
```

```
    int a = 5;
```

```
    int b = 3;
```

```
    int c = 2;
```

```
    int d = a + b * c - !a && c++ % a;
```

```
int d =(a + (b*c) - (!a))&& ((c++)%a);
```

```
    std::cout << "d = " << d << std::endl;
```

```
    std::cout << "c after evaluation = " << c << std::endl;
```

```
    return 0;
```

```
}
```